

RESERVIO

Hotel Management SaaS Platform
Software Requirements Specification (SRS)

Chapters 1 & 2 — Introduction & Overall Description

Draft v1.0 • March 2026

Table of Contents

	2
1. Introduction	3
1.1 Project Overview	3
1.2 Purpose of This Document	3
1.3 Scope	3
1.4 Definitions, Acronyms & Abbreviations	4
1.5 References	5
1.6 Document Overview	5
2. Overall Description	6
2.1 Product Perspective	6
2.2 Product Features	6
2.3 User Classes & Characteristics	8
Guest	8
Receptionist / Staff	8
Housekeeper	9
Hotel Administrator	9
Super Administrator	9
2.4 Operating Environment	10
2.5 Design & Implementation Constraints	10
Multi-Tenancy as a Core Constraint	10
Single Programming Language	11
Free-Tier Infrastructure	11
No Payment Gateway	11
Browser-Only Delivery	11
Stateless Authentication	11
2.6 Assumptions & Dependencies	11
Assumptions	11
Dependencies	12

1. Introduction

1.1 Project Overview

Reservio is a multi-tenant hotel management Software-as-a-Service (SaaS) platform being developed as a 3rd year Bachelor's software engineering project. The platform is designed to allow any hotel — boutique, chain, hostel, or resort — to subscribe and immediately access a fully operational hotel management workspace, without any installation or technical setup.

Hotels today typically rely on a fragmented set of tools: one system for reservations, another for housekeeping coordination, another for guest billing. Reservio consolidates all of these into a single cloud-hosted, subscription-based platform. Each subscribing hotel receives its own isolated workspace — with its own staff accounts, room inventory, guest data, and configuration — all hosted on shared infrastructure managed by the Reservio platform.

The system supports five distinct user roles: Guest, Receptionist/Staff, Housekeeper, Hotel Administrator, and Super Administrator. Each role has its own dedicated interface and permissions, all served from the same codebase and differentiated through role-based access control enforced at the API layer.

1.2 Purpose of This Document

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for Reservio. It serves as the primary reference for the development team, the project supervisor, and any other stakeholder who needs to understand what the system does, how it is structured, and what constraints it operates under.

This document is a first draft covering Chapters 1 and 2 — Introduction and Overall Description. Subsequent chapters will document specific functional requirements, external interface requirements, system attributes, and additional appendices as the project progresses.

**N
O
T
E**

This is a draft document submitted for lab review. Content may be updated following teacher feedback before the final version is published to the project repository.

1.3 Scope

Product Name: Reservio

Product Type: Multi-tenant SaaS web application

Delivery Format: Browser-based, mobile-first, hosted on Railway

Reservio will provide the following high-level capabilities:

- Guest self-service — room browsing, availability search, reservation booking, parking and sunbed reservation, in-app service ordering, and real-time notifications
- Staff operations — reservation management, guest check-in and check-out, room status control, and housekeeping priority overrides
- Housekeeping management — auto-prioritised task queue with real-time updates and room completion marking
- Hotel administration — room and staff account management, dynamic pricing configuration, analytics dashboard, and parking lot management
- Platform administration — tenant onboarding, subscription plan management, and platform-wide analytics for the Reservio Super Admin

The following are explicitly out of scope for the current version:

- Payment processing — billing and subscription payments are referenced conceptually but no payment gateway (e.g. Stripe) will be integrated in the academic version

- Native mobile applications — the system is mobile-responsive but delivered as a web application only
- Third-party channel integrations — no connection to booking platforms such as Booking.com or Expedia
- AI features — a virtual concierge chatbot and anomaly detection service are proposed but not committed; their inclusion is subject to teacher approval and Sprint 5 capacity

1.4 Definitions, Acronyms & Abbreviations

Term	Definition
SaaS	Software-as-a-Service — software delivered over the internet on a subscription basis, hosted by the provider
Multi-tenancy	A software architecture where a single instance of the application serves multiple customers (tenants), with each tenant's data kept isolated from others
Tenant	A subscribing hotel organisation that has its own isolated workspace within the Reservio platform
tenantId	A unique identifier assigned to each tenant, used to scope all database queries and ensure data isolation
JWT	JSON Web Token — a compact, URL-safe token used for stateless authentication; Reservio's JWTs carry userId, tenantId, and role
SRS	Software Requirements Specification — this document
SPA	Single Page Application — a web application that loads a single HTML page and dynamically updates content without full page reloads
API	Application Programming Interface — the set of HTTP endpoints through which the frontend communicates with the backend
REST	Representational State Transfer — the architectural style used for Reservio's API design
ORM	Object-Relational Mapper — a tool that maps database tables to programming language objects; Reservio uses Prisma as its ORM
RBAC	Role-Based Access Control — restricting system access based on the roles assigned to individual users
RAG	Retrieval-Augmented Generation — an AI technique that retrieves relevant context from a knowledge base and passes it to a language model to generate grounded responses (proposed feature)
WebSocket	A communication protocol providing full-duplex communication over a single TCP connection; used in Reservio for real-time notifications
CI/CD	Continuous Integration / Continuous Deployment — automated pipelines that build, test, and deploy code on every push
NFR	Non-Functional Requirement — a requirement that describes how the system performs rather than what it does
US	User Story — a short description of a feature from the perspective of a user (e.g. US-01, US-22)
Housekeeper	A hotel staff member responsible for cleaning and maintaining rooms; assigned tasks automatically by the system
Super Admin	A Reservio platform-level administrator who manages all tenant accounts, not affiliated with any single hotel
Feature Gate	A mechanism that restricts access to certain features based on the tenant's subscription tier (Basic, Premium, or Custom)
Lock (5-minute)	A temporary reservation hold placed on a parking spot or sunbed upon selection, preventing other guests from booking the same spot for 5 minutes

1.5 References

Reference	Description	Location
Reservio Project Proposal	Full project proposal including SaaS business model, architecture, user stories, tech stack, team roles, and sprint plan	GitHub repository — /docs/proposal
Updated_User_Stories.pdf	Original user story document (US-01 to US-21) as submitted prior to proposal development	GitHub repository — /docs
IEEE Std 830-1998	IEEE Recommended Practice for Software Requirements Specifications — structural reference for this document	IEEE Standards
Prisma Documentation	Official documentation for the Prisma ORM used for database access and tenant isolation	prisma.io/docs
React Documentation	Official documentation for the React frontend framework	react.dev
Socket.io Documentation	Official documentation for the WebSocket library used for real-time features	socket.io/docs
Railway Documentation	Official documentation for the hosting and deployment platform	docs.railway.app

1.6 Document Overview

This SRS is structured as follows:

Chapter	Title	Contents
1	Introduction	Project overview, purpose, scope, definitions, references, and this document overview (current chapter)
2	Overall Description	Product perspective and context, high-level feature summary, user classes, operating environment, constraints, and assumptions
3 (upcoming)	Specific Requirements	Detailed functional requirements mapped to user stories, external interface requirements, and system behaviour specifications
4 (upcoming)	Non-Functional Requirements	Performance, security, reliability, scalability, usability, and maintainability requirements with measurable criteria
5 (upcoming)	Appendices	Entity-relationship diagrams, data flow diagrams, API endpoint reference, and glossary expansions

2. Overall Description

2.1 Product Perspective

Reservio is a new, self-contained SaaS product. It does not replace or extend any existing system — it is built from scratch as a cloud-native alternative to traditional property management software (PMS) such as Opera, Mews, or Cloudbeds, which are either too expensive, too complex, or not designed for multi-property SaaS delivery.

The system sits at the intersection of three categories of software: hotel property management, SaaS platform infrastructure, and guest self-service portals. No existing component or legacy system is being reused. All parts — frontend, backend, database, and real-time layer — are built and owned by the project team.

From a SaaS infrastructure perspective, Reservio operates as a shared-infrastructure, isolated-data multi-tenant system. All hotels share the same application codebase and PostgreSQL database instance. Tenant isolation is enforced at the data layer through a `tenantId` column on every database table, and at the API layer through middleware that automatically scopes every query to the current tenant. This architecture is sometimes called row-level multi-tenancy.

The diagram below illustrates the high-level system context — how Reservio sits between hotel operators (tenants), their guests, and the Reservio platform team (Super Admin):

Actor	Relationship to Reservio	Access Point
Hotel Guest	End user of a hotel's Reservio-powered portal	hotel.reservio.com — Guest Portal
Hotel Staff / Receptionist	Operator of the reservation and check-in/out system	hotel.reservio.com — Staff Dashboard
Housekeeper	Operator of the automated task management system	hotel.reservio.com — Housekeeper View
Hotel Administrator	Owner and configurator of the hotel's Reservio workspace	hotel.reservio.com — Admin Panel
Super Admin (Reservio team)	Platform operator — manages all tenant accounts and subscriptions	admin.reservio.com — Super Admin Portal
Reservio Platform	Shared infrastructure serving all tenants	Hosted on Railway; wildcard subdomain *.reservio.com

2.2 Product Features

The following table summarises the high-level features of Reservio. Each feature maps to one or more user stories documented in the project proposal. Detailed functional requirements for each feature will be specified in Chapter 3.

Feature	Description	Subscription Tier	Stories
Guest Account Management	Guests can register, log in, and manage their profile and reservation history on a hotel's Reservio portal	All tiers	US-01, US-02
Room Browsing & Availability	Guests can browse room listings with photos, descriptions, and pricing, and filter by check-in/check-out dates	All tiers	US-03, US-04
Room Reservation	Guests can book a room for selected dates; the system validates availability in real time and confirms the booking	All tiers	US-05

Feature	Description	Subscription Tier	Stories
Parking Spot Reservation	Guests can select a parking spot from a visual lot map; the system applies a 5-minute concurrency lock to prevent double-booking	Premium, Custom	US-06
Sunbed Reservation	Guests can select a sunbed from a visual pool map for a chosen time slot; same 5-minute lock mechanic as parking	Premium, Custom	US-22
Guest Service Marketplace	Guests can order hotel services (spa, room service, airport transfer) through an in-app marketplace; costs are billed to their room	Premium, Custom	US-07
Early Check-In Notification	Guests receive a real-time WebSocket notification when their room has been cleaned and is ready for early check-in	Premium, Custom	US-08
Staff Reservation Dashboard	Staff can view, filter, and manage all hotel reservations; confirm pending bookings; and process guest check-in and check-out	All tiers	US-09 to US-12
Room Status Management	Staff can update room status (Available, Occupied, Maintenance) with changes broadcast in real time across all staff devices	All tiers	US-13
Housekeeping Priority Override	Staff can flag a room's housekeeping task as urgent; the housekeeper's task list re-sorts immediately via WebSocket	All tiers	US-14
Auto-Prioritised Housekeeping Queue	Housekeepers receive a task list automatically sorted by check-out time, urgency, and on-site guest status; updates in real time	All tiers	US-20
Room Cleaning Confirmation	Housekeepers can mark a room as cleaned; the front desk is notified instantly and the guest receives an early check-in alert if applicable	All tiers	US-21
Room Inventory Management	Hotel Admins can add, edit, and remove rooms with full details (type, description, price, photos)	All tiers	US-15
Staff Account Management	Hotel Admins can create, update, and deactivate staff accounts and assign role-based permissions	All tiers	US-16
Dynamic Pricing Engine	Hotel Admins can configure occupancy-based pricing rules (e.g. +20% above 80% occupancy); rules are evaluated and applied automatically by a background cron job	Premium, Custom	US-17
Analytics Dashboard	Hotel Admins can view total bookings, daily revenue, and occupancy percentage with date-range filtering and report export	Premium, Custom	US-18
Parking Lot Management	Hotel Admins can add/remove parking spots, set per-spot pricing, and view occupancy	All tiers	US-19
Tenant Account Management	Super Admins can view, approve, suspend, and delete hotel tenant accounts across the platform	Platform only	US-23
Subscription Plan Management	Super Admins can upgrade or downgrade a tenant's plan; feature gates update immediately	Platform only	US-24

Feature	Description	Subscription Tier	Stories
Platform-Wide Analytics	Super Admins can view platform-level metrics: total tenants, total bookings, and revenue by plan tier	Platform only	US-25
Manual Hotel Onboarding	Super Admins can manually create a tenant account and provision a subdomain; the hotel admin receives an invitation email	Platform only	US-26

2.3 User Classes & Characteristics

Reservio serves five distinct user classes. Each class has different goals, technical expertise levels, and interaction patterns with the system. Access to features is determined by the user's assigned role, which is encoded in their JWT at login and enforced by the API's roleGuard middleware.

Guest

Guests are hotel customers who interact with Reservio through a hotel's branded self-service portal. They are the most numerous user class. Guests are assumed to have no technical knowledge beyond standard web browsing. They interact primarily with the booking flow, maps, and marketplace. Their experience is mobile-first and requires no training.

Attribute	Detail
Technical level	Non-technical — consumer web user
Primary goals	Find and book a room, reserve amenities, order services, receive notifications
Session pattern	Short, task-focused sessions — browse, book, confirm
Device	Primarily mobile; desktop supported
Tenant scope	Scoped to the hotel they registered with; cannot access other hotels' data

Receptionist / Staff

Hotel staff are employees of a subscribing hotel who manage the day-to-day reservation workflow. They are assumed to have moderate computer literacy and will receive brief onboarding from the Hotel Admin. Staff interact with Reservio daily during their shifts, primarily through the reservation dashboard and check-in/out screens.

Attribute	Detail
Technical level	Moderate — comfortable with web-based tools
Primary goals	Confirm reservations, process check-ins and check-outs, update room status, flag urgent housekeeping
Session pattern	Extended sessions during shifts; frequent dashboard checks
Device	Desktop or tablet at front desk
Tenant scope	Scoped to their hotel tenant; cannot access other tenants' reservations

Housekeeper

Housekeepers are hotel staff responsible for cleaning and maintaining rooms. They interact with Reservio exclusively through the housekeeper task board, which is designed to be extremely simple — a mobile-first list of rooms to clean, sorted automatically by priority. No configuration or complex navigation is required.

Attribute	Detail
Technical level	Basic — designed for minimal training

Attribute	Detail
Primary goals	View task list, see which rooms are highest priority, mark rooms as cleaned
Session pattern	Short, frequent check-ins throughout their shift
Device	Mobile phone
Tenant scope	Scoped to their hotel tenant; only sees their assigned rooms

Hotel Administrator

Hotel Administrators are the primary account owners for a subscribing hotel. They are responsible for configuring and maintaining the hotel's Reservio workspace — adding rooms, managing staff, setting pricing rules, and monitoring performance. They are assumed to be business owners or managers with moderate technical confidence.

Attribute	Detail
Technical level	Moderate — business-oriented, not developer-level
Primary goals	Configure the hotel workspace, manage staff and inventory, monitor revenue and occupancy, adjust pricing
Session pattern	Regular but not constant — configuration sessions and daily performance checks
Device	Desktop
Tenant scope	Full access to their own tenant's data and configuration; cannot access other tenants
Feature access	Determined by subscription tier (Basic, Premium, or Custom)

Super Administrator

The Super Administrator is a member of the Reservio product team — not affiliated with any hotel. This role has cross-tenant visibility and is responsible for managing the health and growth of the platform itself: onboarding new hotels, managing subscriptions, and monitoring platform-wide metrics. There will typically be one or two Super Admins in production.

Attribute	Detail
Technical level	High — internal Reservio team member
Primary goals	Approve and manage tenant accounts, control subscription plans, monitor platform health
Session pattern	Occasional — on-demand when hotel onboarding or plan changes are needed
Device	Desktop
Tenant scope	Cross-tenant — can read and modify any tenant's account metadata (not guest data)
Access point	admin.reservio.com — separate from hotel-facing subdomains

2.4 Operating Environment

Reservio is a cloud-hosted web application. The following describes the environment in which the system will operate:

Component	Environment	Detail
Hosting platform	Railway	Cloud PaaS; supports wildcard subdomains (*.reservio.com) for tenant routing; auto-deploys from GitHub main branch
Frontend runtime	Modern web browser	Chrome, Firefox, Safari, Edge — latest two major versions. No browser plugin or installation required.
Frontend framework	React 18 + Vite	Served as a compiled SPA; all role-based views from one codebase
Backend runtime	Node.js 20 LTS	Express.js API server; runs as a single Railway service
Database	PostgreSQL 16	Managed PostgreSQL instance on Railway; single instance shared across all tenants with row-level isolation
Real-time layer	Socket.io v4	WebSocket server co-located with Express; tenant-scoped rooms
Background jobs	node-cron v3	Runs within the Node.js process; evaluates pricing rules and (if approved) anomaly detection nightly
CI/CD pipeline	GitHub Actions	Triggers on push to main; runs ESLint, Jest tests, then deploys to Railway via deploy hook
Network	HTTPS	All traffic encrypted via TLS; Railway provides SSL termination with auto-renewing certificates
Device support	Desktop & mobile	Responsive design using Tailwind CSS; guest-facing screens designed mobile-first at 375px minimum width

2.5 Design & Implementation Constraints

The following constraints shape the design and implementation decisions for Reservio. These are not preferences — they are fixed boundaries the system must operate within.

Multi-Tenancy as a Core Constraint

The system is designed multi-tenant from the ground up. This is not an optional feature — it is the foundational architectural principle. Every database table must have a `tenantId` column. Every API query must be scoped to a tenant. The Prisma client extension that auto-injects `tenantId` must be used for all database access; direct Prisma client calls that bypass tenant scoping are forbidden in the codebase.

Single Programming Language

The entire system — frontend and backend — must be implemented in JavaScript (with optional TypeScript). This constraint was chosen to reduce the onboarding burden for a 7-person team. The only exception is the proposed AI anomaly detection feature, which would use a Python microservice; this is explicitly flagged as an exception and is not committed scope.

Free-Tier Infrastructure

The system must operate within the free or low-cost tiers of its hosting and tooling. Railway's free tier imposes memory and compute limits. The database must fit within Railway's free PostgreSQL instance limits for the duration of the project. This constrains the number of concurrent active tenants the system can realistically support during the academic delivery period.

No Payment Gateway

Payment processing is out of scope. The subscription tier model is implemented at the data and middleware level (plan field on the tenant record), but no actual billing, invoicing, or card processing will be integrated. This is explicitly noted in the scope section and must be communicated clearly to the teacher and any stakeholders evaluating the project.

Browser-Only Delivery

Reservio is delivered as a web application only. No native iOS or Android application will be built. The guest-facing interface must be fully functional on mobile browsers at a minimum viewport width of 375px (iPhone SE size), achieved through responsive design using Tailwind CSS utility classes.

Stateless Authentication

Authentication must be stateless, implemented using JSON Web Tokens (JWT). No server-side session store (e.g. Redis) will be used. This simplifies horizontal scaling and removes the need for session synchronisation across processes. JWT expiry and refresh logic must be handled client-side.

2.6 Assumptions & Dependencies

Assumptions

The following assumptions have been made during the design of Reservio. If any of these assumptions proves incorrect, requirements or design decisions may need to be revisited.

ID	Assumption	Impact if Invalid
A-01	Hotels subscribing to Reservio have a stable internet connection sufficient for real-time WebSocket communication	Real-time features (notifications, task updates) would be unavailable or unreliable for affected hotels
A-02	All users (guests and staff) access Reservio through a modern web browser with JavaScript enabled	The SPA would not function; a server-rendered fallback is not planned
A-03	Railway's wildcard subdomain feature (*.reservio.com) will remain available and functional throughout the project	Tenant routing would need to fall back to path-based routing (e.g. reservio.com/hotel-name)
A-04	The team will have access to a Railway account with sufficient free-tier capacity for development and demo	Hosting would need to migrate to an alternative platform such as Render or Fly.io
A-05	Each hotel tenant has at most one Hotel Administrator account at initial setup; additional admin accounts can be added later	The onboarding flow and invitation logic would need to support multiple simultaneous admin accounts from the start
A-06	Guests register with and belong to a single hotel tenant; cross-hotel guest accounts are not required in this version	A unified guest identity system across tenants would require significant additional architecture
A-07	The dynamic pricing engine is evaluated nightly by the cron job; real-time sub-minute evaluation is not required	A more frequent evaluation schedule would require a message queue or event-driven architecture beyond the current stack
A-08	If AI features are approved, the team will have access to an OpenAI API key for the chatbot RAG feature, or will use a self-hosted open-source model (Mistral 7B)	The chatbot feature could not be implemented without access to a language model API or sufficient server resources for a local model

Dependencies

The following external dependencies are required for Reservio to function. Risks associated with each dependency are documented in the project proposal's risk register.

Dependency	Type	Version / Tier	Purpose	Risk
Railway	Hosting platform	Free tier	Application hosting, PostgreSQL database, wildcard subdomain support	Medium — free tier limits; wildcard subdomain availability
GitHub + GitHub Actions	Version control & CI/CD	Free tier	Source code management, automated build and deploy pipeline	Low — widely used, highly reliable

Dependency	Type	Version / Tier	Purpose	Risk
Node.js	Runtime	v20 LTS	Backend JavaScript execution environment	Low — LTS release, stable
React	Frontend framework	v18	UI component model and rendering	Low — major stable release
PostgreSQL	Database	v16	Primary data store; all tenant data	Low — mature, reliable
Prisma	ORM	v5	Database access, schema management, tenant isolation via client extension	Low — actively maintained
Socket.io	Real-time library	v4	WebSocket communication for live features	Low — widely used
node-cron	Scheduler	v3	Background job execution for pricing engine	Low — lightweight, no external dependency
OpenAI API	External AI service	GPT-4o-mini	Language model for chatbot feature (if approved)	Medium — requires API key; usage costs may apply
pgvector	PostgreSQL extension	Latest	Vector similarity search for chatbot RAG (if approved)	Low-Medium — must be available on Railway PostgreSQL instance